دانشگاه آزاد اسلامی – واحد زنجان
سند معماری -رستوران یار
سیستم مدیریت رستوران (RestaurantYar)
نویسنده: مبینا قرابیگلی
پروژه درس:مهندسی نرم افزار
نام استاد:آقای دکتر ابراهیم فضلی
روز و ساعت کلاس: دوشنبه ساعت۱۰:۴۰
رشته:مهندسی کامپیوتر
نیمسال اول سال تحصیلی ۱۴۰۵-۱۴۰۶
نویسنده این بخش: مبینا قرابیگلی
تایپ متن پروژه: مبینا قرابیگلی

سند معماری_رستوران یار

اطلاعات پروژه 🍴

نام پروژه	رستوران‌یار (RestaurantYar)
نسخهٔ سند	۱.۰
تاریخ	۱۴۰۵/۰۷/۲۶
تهیه‌کنندگان	مبینا قرابیگلی، آیلین کاوسیان ، یگانه کریم زاده
وضعیت	پیش‌نویس

۱. مقدمه

۱.۱ هدف سند

این سند معماری نرم‌افزار پروژهٔ رستوران‌یار را تعریف می‌کند. هدف آن ایجاد یک درک مشترک از ساختار سیستم، تصمیم‌های کلیدی و محدودیت‌ها است.

۱.۲ دامنه

- در دامنه: معماری سطح بالا، لایه‌ها، Endpointها، امنیت، جریان داده.
- خارج از دامنه: جزئیات پیاده‌سازی، کد، طراحی UI.

۱.۳ ذی‌نفعان

ذی‌نفعان پروژه

ذی‌نفع	نقش
تیم توسعه	پیاده‌سازی
استاد درس	ارزیابی
مشتری فرضی (آقای کریمی)	تأیید

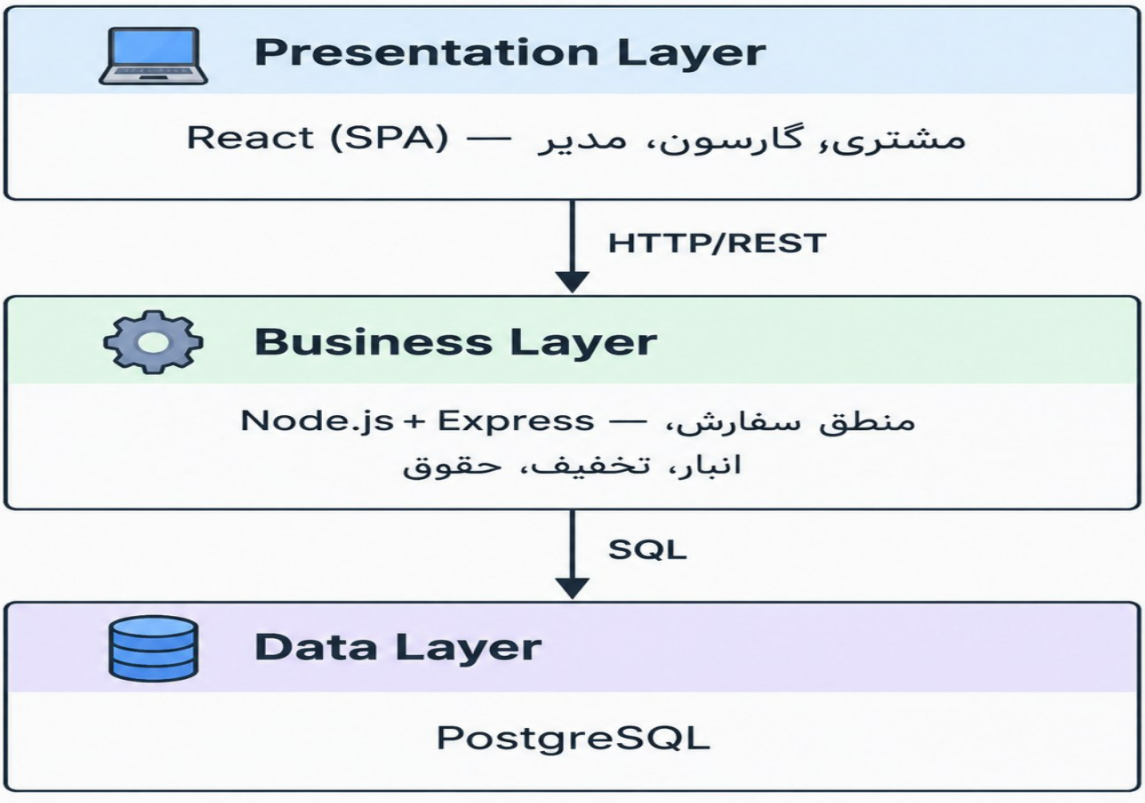
۲. نمای کلی معماری

۲.۱ سبک معماری

Monolith + Layered Architecture

- یک برنامهٔ واحد
- سه لایهٔ اصلی: Presentation، Business، Data
- مناسب تیم کوچک (۳ نفر)
- ساده برای شروع، قابل گسترش

۲.۲ دیاگرام لایه‌ها



۲.۳ جریان داده (نمونه: ثبت سفارش)

1. مشتری از طریق React روی QR میز، آیتم‌ها را انتخاب می‌کند
2. Request به /api/orders ارسال می‌شود
3. Business Layer اعتبارسنجی و محاسبهٔ هزینه (شامل تخفیف و ارسال) انجام می‌دهد
4. موجودی مواد اولیه در انبار کاهش می‌یابد
5. سفارش در پایگاه داده ثبت می‌شود
6. اعلان به آشپزخانه و پیک ارسال می‌شود
7. پاسخ موفقیت به مشتری برگردانده می‌شود

۳. لایه‌ها

۳.۱ Presentation Layer (Frontend)

🚀 مجموعه فناوری‌های پروژه

مورد انتخاب	فناوری
فناوری	React 18
Routing	React Router
State	Redux Toolkit
UI	Tailwind CSS
Build	Vite
ارتباط با API	Axios

نکته: رابط کاربری باید واکنش‌گرا (Responsive) باشد چون مشتری، گارسون، آشپز و پیک از دستگاه‌های مختلف استفاده می‌کنند.

۳.۲ Business Layer (Backend)

⚙️ فناوری‌های اصلی پروژه

انتخاب	مورد
Node.js 20 + Express 4	فناوری
Prisma	ORM
Zod	Validation
JWT + bcrypt	Auth
Winston	Logging
Socket.io	اعلان بلادرنگ

۳.۳ Data Layer (Database)

۴. تصمیم‌های معماری (ADR)

ADR-01: چرا Monolith؟

تصمیم: معماری Monolith

دلیل:

· تیم کوچک (۳ نفر)

· پیچیدگی کم

· سرعت توسعهٔ بالاتر

· استقرار ساده

· برای MVP کافی است و در آینده می‌توان به Microservices مهاجرت کرد

ADR-02: چرا React؟

تصمیم: React برای Frontend

دلیل:

· محبوب و مستند

· اکوسیستم بزرگ

· SPA برای UX بهتر

· مناسب برای رابط چندنقشی (مشتری، گارسون، آشپز، مدیر)

ADR-03: چرا PostgreSQL؟

تصمیم: PostgreSQL برای Database

دلیل:

· Relational و قدرتمند

· پشتیبانی از JSON برای تنظیمات پویا (مثل فرمول غذا و قوانین تخفیف)

· رایگان و اوپن‌سورس

· پشتیبانی از تراکنش برای عملیات مالی (پرداخت، حقوق)

برای اعلان؟ Socket.io چرا :ADR-04
تصمیم: Socket.io برای اعلان‌های بلادرنگ
دلیل:
· نمایشگر آشپزخانه باید سفارش جدید را بلافاصله ببیند
· پیک باید اعلان آماده بودن سفارش را بلافاصله بگیرد
· مشتری باید تغییر وضعیت سفارش را بلادرنگ ببیند
برای رمز عبور؟ bcrypt چرا :ADR-05
تصمیم: bcrypt برای هش رمز عبور
دلیل:
· استاندارد صنعتی برای ذخیرهٔ امن رمز
· نمک تصادفی برای هر کاربر
· ضریب هزینهٔ قابل تنظیم (۱۲)
· مقاوم در برابر حملات بروت‌فورس و Rainbow Table

۵. REST API.

۵.۱ Endpoint(های عمومی (بدون احراز هویت

🚀 جدول Endpoint های API

Method	Endpoint	توضیح
GET	/api/menu	لیست منو با دسته‌بندی
GET	/api/menu/:id	جزئیات یک غذا
GET	/api/categories	لیست دسته‌بندی‌ها
GET	/api/menu/search	جستجو در منو
POST	/api/auth/register	ثبت‌نام
POST	/api/auth/login	ورود

۵.۲ Endpointهای مشتری.

لیست API درخواست‌ها

Method	Endpoint	توضیح
POST	/api/orders	ثبت سفارش جدید
GET	/api/orders/:id	جزئیات سفارش
GET	/api/orders/:id/status	پیگیری وضعیت سفارش
POST	/api/tables/reserve	رزرو میز
POST	/api/discount/validate	اعتبارسنجی کد تخفیف
POST	/api/orders/:id/review	ثبت نظر و امتیاز

۵.۳ Endpointهای گارسون

API Endpoints

Method	Endpoint	توضیح
GET	/api/tables	وضعیت میزها
POST	/api/orders/waiter	ثبت سفارش حضوری
PUT	/api/orders/:id/assign-table	اتصال سفارش به میز

۵.۴ Endpointهای آشپزخانه (KDS)

API های بخش آشپزخانه

Method	Endpoint	توضیح
GET	/api/kitchen/orders	سفارش‌های در انتظار
PUT	/api/kitchen/orders/:id/status	تغییر وضعیت (در حال آماده‌سازی، آماده)

۵.۵ Endpointهای پیک.

🚚 · API های مدیریت سفارشات تحویل

Method	Endpoint	توضیح
GET	/api/delivery/orders	سفارش‌های آماده برای ارسال
PUT	/api/delivery/orders/:id/accept	پذیرش سفارش
PUT	/api/delivery/orders/:id/delivered	ثبت تحویل

۵.۶ Endpointهای انبارداری

📦 نقاط پایانی API موجودی انبار

Method	Endpoint	توضیح
GET	/api/inventory	لیست موجودی مواد اولیه
POST	/api/inventory	ثبت ورود مواد
PUT	/api/inventory/:id	به‌روزرسانی موجودی
GET	/api/inventory/low-stock	لیست مواد کم‌موجود

۵.۷ Endpointهای مدیر (Admin)

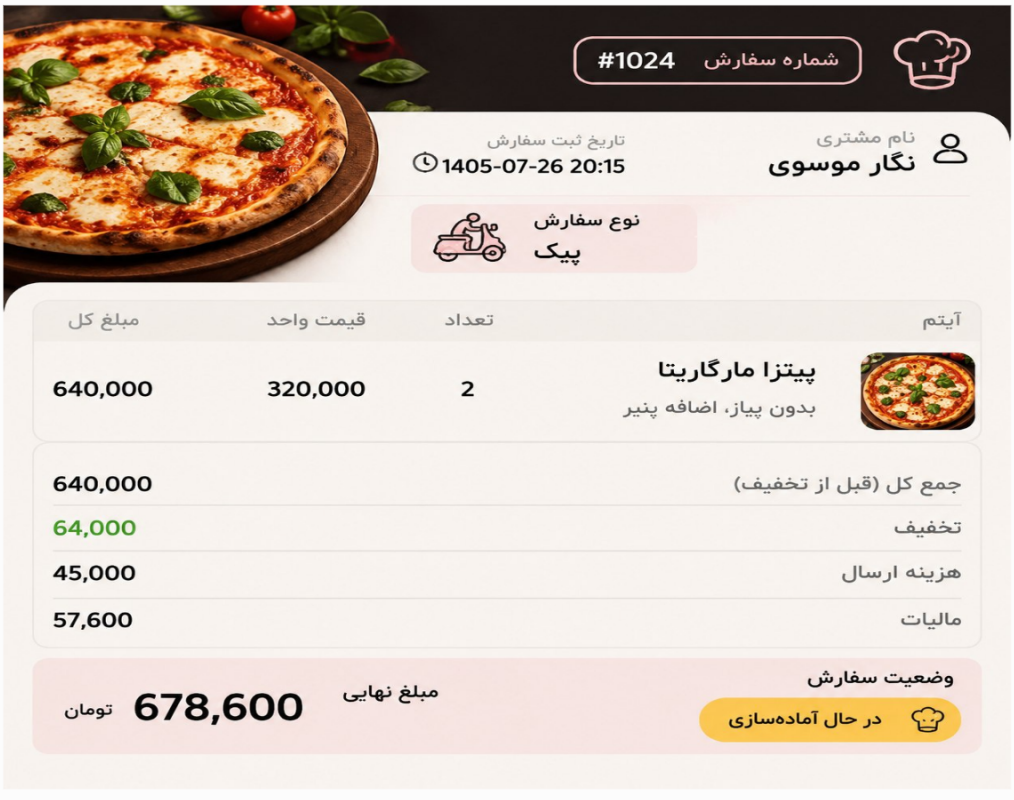
لیست API های ادمین ⚙️

Method	Endpoint	توضیح
POST	/api/admin/menu	افزودن غذا
PUT	/api/admin/menu/:id	ویرایش غذا
DELETE	/api/admin/menu/:id	حذف غذا
POST	/api/admin/discount	تعریف کد تخفیف
GET	/api/admin/orders	لیست همهٔ سفارش‌ها
PUT	/api/admin/orders/:id/status	تغییر وضعیت سفارش
POST	/api/admin/employees	افزودن کارمند
POST	/api/admin/payroll/calculate	محاسبهٔ حقوق
GET	/api/admin/reports/sales	گزارش فروش
GET	/api/admin/reports/delivery	گزارش هزینهٔ ارسال
GET	/api/admin/reports/payroll	گزارش حقوق

۵.۸ نمونه Response (سفارش)


```
}
"id": 1024,
"tableNumber": 7,
"customerName": "نگار موسوی",
"orderType": "delivery",
"items": [
    }
"menuId": 12,
"name": "پیتزا مارگاریتا",
"quantity": 2,
"unitPrice": 320000,
"customization": "بدون پیاز، اضافه پنیر"
    {
```

,[
"subtotal": 640000,
"discount": 64000,
"deliveryFee": 45000,
"tax": 57600,
"total": 678600,
"status": "preparing",
"createdAt": "1405-07-26T20:15:00"
{



۶. امنیت

۶.۱ احراز هویت

- JWT برای API
- bcrypt برای رمزنگاری گذرواژه (SALT_ROUNDS = 12)
- Refresh Token برای تمدید

۶.۲ مجوزدهی

· Role-Based Access Control (RBAC)
· نقش‌ها: Customer، Waiter، Chef، Cashier، Warehouse، Delivery، Admin
· هر نقش فقط به Endpointهای مربوط به خود دسترسی دارد

۶.۳ امنیت API

· HTTPS اجباری
· Rate Limiting (۱۰۰ درخواست در دقیقه)
· CORS محدود به دامنهٔ فرانت‌اند
· Input Validation با Zod
· ثبت لاگ فعالیت با Winston

۶.۴ امنیت تراکنش‌های مالی

· محاسبهٔ هزینه‌ها در سمت سرور، نه کلاینت
· استفاده از تراکنش دیتابیس برای ثبت سفارش و کاهش موجودی
· جلوگیری از Race Condition در استفادهٔ همزمان از کد تخفیف

۷. مقیاس‌پذیری

۷.۱ محدودیت‌های فعلی

· Monolith → مقیاس‌پذیری محدود
· یک سرور → تک نقطهٔ شکست

۷.۲ مسیر رشد

1. فاز ۱ (MVP): Monolith + یک سرور
2. فاز ۲: Load Balancer + چند سرور
3. فاز ۳: Database Replication
4. فاز ۴: جدا کردن سرویس‌های پرمصرف (KDS، اعلان‌ها)
5. فاز ۵: مهاجرت به Microservices

۸. ریسک‌ها و بدهی فنی

جدول مدیریت ریسک 🛡️

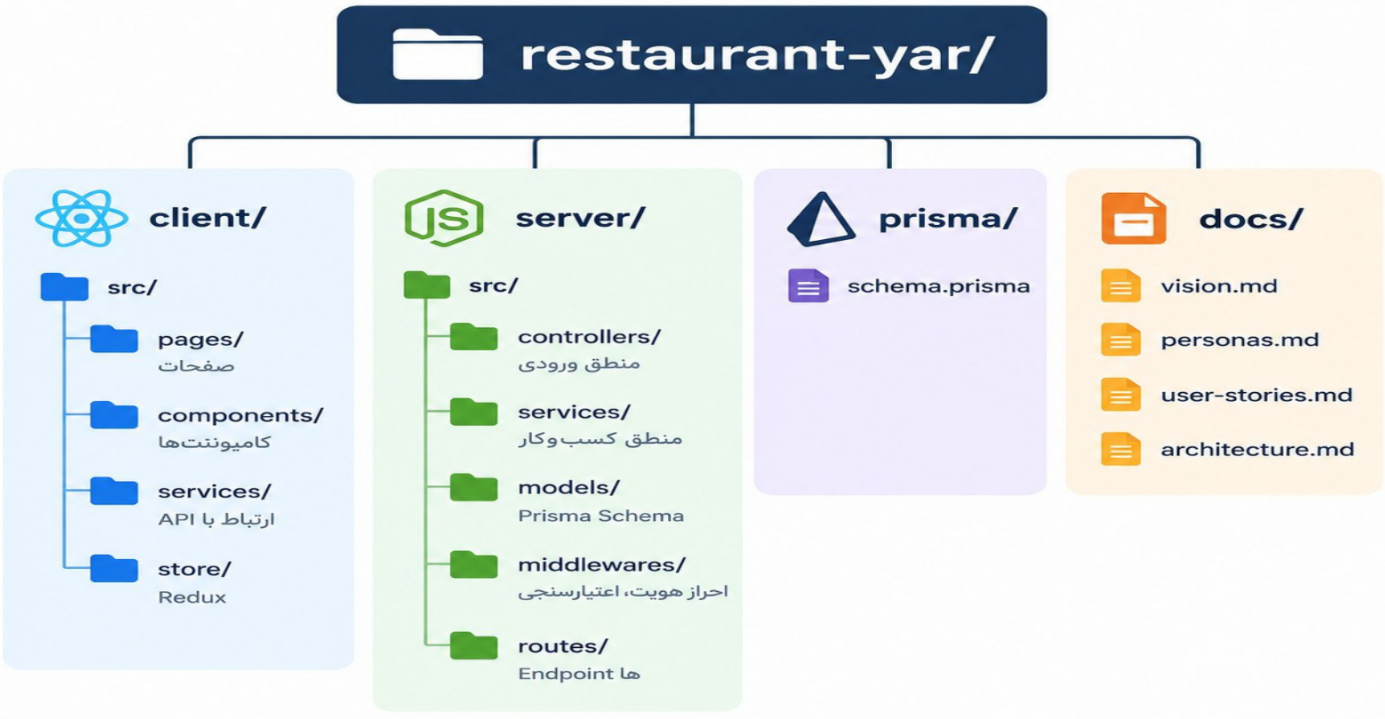
ریسک	احتمال	اثر	راهکار
ازدحام سفارش در ساعت شلوغی	بالا	متوسط	صف پیام (Redis + Bull)
نفوذ امنیتی	کم	بالا	رعایت OWASP Top 10
از دست دادن داده	کم	بالا	Backup روزانه
قطعی اینترنت	متوسط	بالا	Cache محلی + حالت آفلاین
محاسبهٔ نادرست هزینه ارسال	متوسط	بالا	تست Unit دقیق
محاسبهٔ نادرست حقوق	متوسط	بالا	تست Unit دقیق
بدهی فنی	بالا	متوسط	Refactor در Sprint 3

۹. فناوری‌ها

لایه فناوری 💻

لایه	فناوری
Frontend	React 18 + Vite + Tailwind + Redux Toolkit
Backend	Node.js 20 + Express 4
Database	PostgreSQL 16
ORM	Prisma
Auth	JWT + bcrypt (SALT_ROUNDS=12)
Real-time	Socket.io
Testing	Jest + Supertest
CI/CD	GitHub Actions
Deploy	Railway

۱۰. ساختار پروژه (پیشنهادی)



۱۱. تست

انواع تست‌های نرم‌افزار

سطح	ابزار	هدف
Unit Test	Jest	تست منطق محاسبات (هزینه ارسال، تخفیف، حقوق)
Integration Test	Supertest	تست Endpointها
E2E Test	Playwright	تست جریان کامل سفارش

هدف پوشش تست: حداقل ۷۰٪

پیوست الف — تاریخچهٔ تغییرات

نسخه تاریخ تغییرات تهیه‌کننده 📋

نسخه	تاریخ	تغییرات	تهیه‌کننده
۱.۰	۱۴۰۵/۰۷/۲۶	نسخهٔ اولیه	مبینا قرابیگلی